

Mewndo v1: Voice Build Prompts (after the submission)

Speak each prompt to your coding agent (Claude Code, Opus 5.5) through Wispr Flow, one at a time. Each prompt names the spec section it builds, so the agent and you can check the work against `mewndo-product-spec.md`.

Do not start this file before you submit v0. Finish v0 Step 15 (installer) and Step 16 (README) and submit before **6 Oct 2026, 11:59 PM**. Everything here is post-submission work, roughly 6 to 12 months for one person with AI agents (spec §31.1).

Read this first

Where Clef should run (and why not Kaggle for the live model)

Option	Cost	Good for	Not good for
Cloudflare Workers AI (<code>@cf/cloudflare/clef-flash</code>)	Free: 10,000 neurons per day. Clef-flash costs 8,182 neurons	The live decision model for Guard, Send Guard, Heal and voice	Location and latency aren't guaranteed (spec §29.2), so every call

Option	Cost	Good for	Not good for
	per million input tokens, so about 1.2 million tokens a day free , roughly 1,200 decisions a day at 1,000 tokens each. Beyond that, \$0.011 per 1,000 neurons	intents. Same model, hosted, fast, free at beta scale	needs a deadline and fallback
Kaggle (free notebooks)	Free: about 30 GPU hours a week, sessions up to 9 hours, P100 or 2× T4 GPUs, phone verification needed	Training and testing: build Mewndo's evaluation set, benchmark Clef-flash accuracy, fine-tune Laya (the small local	Serving the live model. Sessions stop after 9 hours, there's no permanent server or public endpoint, and a 9B model on a T4 is far

Option	Cost	Good for	Not good for
		judge) and export it	slower than 39 ms (a 4-bit Clef-flash on a 16 GB Mac measured about 2 s median)
Self-hosted Clef-flash on dedicated GPUs (spec §29)	About \$1,300 per month per GPU	The firm ~39 ms promise at scale	Not free; do it only when usage justifies it

So the plan is: Workers AI (free) runs Clef-flash live; Kaggle (free) trains and tests; the desktop app decides most cases with rules locally and later with a fine-tuned Laya on the PC.

The free stack for everything else

Need	Free option	Limit to remember
Hosted MCP server, Send Guard, decision service, cloud workers	Cloudflare Workers (free plan) with the Agents SDK <code>McpAgent</code> and the OAuth provider library	Free-plan request and CPU limits; fine for a beta

Need	Free option	Limit to remember
Per-user state, hold queues, realtime connection to the desktop	Durable Objects (free tier: 100,000 requests a day, 5 GB SQLite storage)	Plenty for a beta
Gmail and Drive access	Google Cloud project in "Testing" mode	Up to 100 test users ; an unverified-app warning; tokens expire every 7 days , so testers reconnect weekly. Public launch needs Google verification (and a security assessment for restricted Gmail scopes)
Gmail push notifications	Google Cloud Pub/Sub, pushing to a Worker URL	Check the free quota in your Google Cloud console
Training GPUs	Kaggle	About 30 h a week
CI on Windows	GitHub Actions (free for public repos)	Minutes limit for private repos

Speaking rules (same as v0)

- One prompt at a time. Wait for the agent to finish, check **You should see**, then paste the agent's summary to me if anything looks off.

- End every step with tests passing, a commit and a push (each prompt says so).
 - Say the name once per session as "**Mewndo, M-E-W-N-D-O**" so Wispr Flow doesn't write "Muendo".
 - If the agent asks a question you're unsure about, paste it to me before answering.
-

Phase 0: Prepare the repo

P0.1 Spec and rules

We're starting Mewndo version 1. The product name is Mewndo, M-E-W-N-D-O. I'm attaching the product spec; save it in the repo as docs/spec.md and treat it as the source of truth. Sections 21 to 31 describe version 1.

Add these rules to CLAUDE.md, keeping all the existing ones. One: the v0 Node engine stays as the reference until the new Rust core passes every v0 test. Two: never weaken or delete an existing test to make something pass. Three: every network call to a model has a deadline and a rule-based fallback, as in spec section 29.4. Four: never store passwords, keys or secret files. Five: every feature states honestly what it can't do, matching spec section 28.10. Six: free tiers first; no paid service without asking me.

Commit with the message "v1 spec and rules" and push.

You should see: docs/spec.md in the repo and the new rules in CLAUDE.md.

After the submission, attaching the spec file to the agent is fine. If you want to keep the whole project voice-only, read the key sections aloud instead.

P0.2 Repo layout

Reorganize the repo into: apps/desktop for the Electron app, core for a new Rust service, cloud for Cloudflare Workers code, notebooks for Kaggle notebooks, and docs. Move the existing code without changing behaviour. Update the scripts so npm test, npm run reliability and npm start still work from the root. Run everything, commit with the message "repo layout for v1" and push.

You should see: all v0 tests and the reliability suite still passing.

Phase 1: Rust core and the lowest-latency path (spec §28.6, §31.2)

P1.1 Core service skeleton

Create the Rust service in the core folder, called mewndo-core. It runs in the background, started by the desktop app, and talks to the app over a Windows named pipe, with a Unix socket on other systems. Define a small typed message protocol with a version number. Add logging to the same rotating log folder the app uses, and automatic restart if it crashes. The desktop app should start it, check it's alive, and show its status. Don't move any engine logic yet. Add tests for the protocol, commit with the message "mewndo-core skeleton" and push.

You should see: the app reports "core running", and killing the core process makes it restart.

P1.2 Content store in Rust

Port the content store to mewndo-core using the exact same on-disk format as v0, so existing history keeps working. Hash

and compress with a worker pool of 8 to 32 threads. Write metadata in batches in one transaction. Keep the temp file suffix and startup cleanup. Write a compatibility test that reads a store created by the v0 Node engine and one that makes the Node engine read a store written by Rust. Run all tests, commit with the message "rust content store" and push.

You should see: both compatibility tests passing.

P1.3 Change feed

Port the watcher to mewndo-core. On Windows use ReadDirectoryChangesW, and use the NTFS USN change journal to catch up on changes made while Mewndo was closed or when the watch buffer overflows, followed by a reconciliation scan. Detect deletes immediately, without waiting for a write to finish, because deletes need no content. Keep the same ignore rules, never follow links or junctions, and keep long path support. Measure how long a delete takes to reach the journal and print it in the test output. Commit with the message "rust change feed" and push.

You should see: delete detection times printed, ideally under 100 ms.

P1.4 Restore ladder

Build the restore engine in mewndo-core following spec section 28.6. Use the restore ladder: first rename from Mewndo's trash if it's on the same volume; then a copy-on-write clone if the volume supports it; then parallel copy with CopyFile2 on Windows, which uses block cloning automatically on Dev Drive and ReFS. Restore into a staging folder on the same volume and rename into place. Flush once at the end, not per file; the restore log already makes a crash resumable. Show the user "restored" when the renames finish,

and run verification in the background, writing the result to the restore log. Keep everything from v0: before-undo save point, trash instead of delete, retries for locked files, crash recovery. Commit with the message "rust restore ladder" and push.

You should see: all restore tests passing against the Rust core.

P1.5 Freeze and resume

Add process control to mewndo-core: freeze and resume a whole process tree on Windows by process ID, and end it if asked. This is for the Brake feature in spec section 24.3. Never touch system processes or Mewndo itself. Add tests that freeze and resume a test process. Commit with the message "process freeze" and push.

P1.6 Benchmarks

Add a benchmark command, `npm run bench`, that measures: restoring one 1 GB file, restoring 5,000 small files, a first scan of 50,000 files, and idle memory and CPU. Run it on NTFS, and on a Dev Drive if one exists, and print a table with times. Run it with Windows Defender on. Targets: 5,000 files under 5 seconds, 1 GB under 1 second on the same volume. If a target is missed, find the cause and fix it, but never by removing a safety step. Save results to `docs/benchmarks.md`. Commit with the message "benchmarks" and push.

You should see: a table in `docs/benchmarks.md`. Paste it to me.

P1.7 Shadow mode and cutover

Run every v0 unit test and the reliability suite against the Rust core. Then add a shadow mode setting in which both engines run on the same folders and compare their manifests every hour, logging any difference. Add a setting to choose the

engine, defaulting to Rust only when all tests pass. Commit with the message "rust core cutover" and push.

You should see: every v0 test and all 27 reliability checks passing on the Rust core.

Phase 2: Guard, Brake, Heal, Resume (spec §24, §31.3)

P2.1 Brief to policy

Build the policy engine in mewndo-core from spec section 24.1. Turn each brief's scope and rules into a policy: allowed folders, files the brief names, delete rules, and secret paths such as .env files, SSH keys and browser profiles. The engine takes a planned action and returns allow, deny or ask, with a short reason an agent can read. Rules must answer in under 5 milliseconds. Cover: delete of a file the brief didn't name, writes outside the scope, recursive or wildcard deletes, git reset hard, git clean, force-push, secret file access, and bursts. Write a test for each rule. Commit with the message "policy engine" and push.

P2.2 Guard for Claude Code

Switch the Claude Code PreToolUse hook to the HTTP hook type pointed at mewndo-core's local server, keeping the token protection. Answer with hookSpecificOutput permissionDecision set to allow, deny or ask, a permissionDecisionReason, and additionalContext telling Claude why and what to do instead. If Mewndo doesn't answer in time, harmless actions pass and destructive ones are denied, with a setting to fail open. In-scope deletes create a

save point before they're allowed. Update the setup button to show the exact change first. Check the current Claude Code hooks documentation before writing the config. Test with a fake hook input for each rule. Commit with the message "guard for claude code" and push.

You should see: a test where an out-of-scope delete is denied with a readable reason.

P2.3 Guard for Codex and Cursor

Add the same Guard for Codex and Cursor. For Codex, write a PreToolUse entry in the user's codex hooks.json; it covers shell commands, apply_patch file edits and MCP calls, and denies with permissionDecision deny or exit code 2. For Cursor, write preToolUse and beforeShellExecution entries in the user's cursor hooks.json, answering with permission deny and an agent_message. Check each tool's current hook documentation first. Show the exact change before writing, keep a backup, and make installing twice change nothing. Test both with fake inputs. Commit with the message "guard for codex and cursor" and push.

P2.4 Brake and Heal

Build Brake and Heal from spec sections 24.3 and 24.4. Brake for Claude Code returns continue false with a stop reason on the next hook. For Codex and Cursor, deny every further action with a stop message until I resume. For any other agent process, freeze it with the process control from P1.5. Heal: when an agent session deletes or changes something outside its brief, restore it automatically once and show a drift card; if the agent does the same thing again, brake instead of fighting it. Never auto-heal changes made while no agent session is active. If the same agent drifts three times in one task, stop

and hand it to me. Commit with the message "brake and heal" and push.

P2.5 Resume with a Continue card

Build Resume from spec section 24.5. When I click Resume after a brake, write a Continue card: the original task, what's done with each item marked Verified from the journal or Agent says, what went wrong, what was healed, and a new rule that prevents it happening again. Inject it: for Claude Code through a SessionStart hook returning additionalContext, resuming the previous session when possible; for Codex through a Mewndo-managed block in AGENTS.md; for Cursor through a Mewndo-managed rules file; for anything else, copy it to the clipboard. Keep cards under 600 tokens. Commit with the message "resume with continue card" and push.

P2.6 Fake agent test harness

Build a test harness with a scripted fake agent that sends hook inputs like Claude Code, Codex and Cursor would. Script these runs: in-scope edits, an out-of-scope delete, a secret file read, a recursive delete, a 300-file burst, and a run that drifts three times. Check the verdict, the brake, the heal and the resume card for each agent type. Add it to npm test. Commit with the message "guard test harness" and push.

You should see: every scripted run giving the expected verdict for all three agents.

Phase 3: The Mewndo bar (spec §23, §31.4)

P3.1 The pill

Build the Mewndo bar from spec sections 23.5 to 23.7: a small dark pill floating above the taskbar, default bottom right so it doesn't sit on top of Wispr Flow's bar at bottom centre. It's frameless, transparent, always on top, never takes focus, has no taskbar entry, and appears without stealing focus. Transparent areas let clicks through; the pill itself takes clicks. One per display, remembers where I drag it, and can dock to either side edge. Resting look: a protection dot, one small dot per running agent, a mic button and an up-arrow bubble. Shrinks to a dot after 5 seconds idle. Hide during full-screen apps and screen sharing unless there's an alert. Test at 100 and 150 percent display scale. Commit with the message "mewndo bar pill" and push.

You should see: the pill above the taskbar; typing in another app is never interrupted.

P3.2 Hover and actions

On hover, expand the pill to show the shortcuts and the change ticker, like minus 12, tilde 5, plus 3 for deleted, edited and created since the last save point, with buttons Undo last, Brake and Save point. Tap the ticker to open the diff; long-press to undo that burst. Tap an agent dot to open that agent's lane; long-press to brake it. All actions work with the main window closed. The bar renders only from mewndo-core's event stream and updates within 100 milliseconds. Commit with the message "mewndo bar actions" and push.

P3.3 Drift card and hold chips

When Guard or Heal fires, expand the bar into the drift card from spec section 23.4: what the agent tried, what Mewndo blocked or restored, and three buttons, Resume with corrected brief, Let it, and Stop and review. Add hold chips with a

countdown, approve and cancel, using local holds for now; cloud holds come in Phase 5. The card appears within 200 milliseconds of the decision. Commit with the message "drift card" and push.

P3.4 The up-arrow panel

Build the up-arrow panel from spec section 23.8. Clicking the up-arrow bubble beside the mic slides open a small panel, about 360 by 420 pixels, in about 150 milliseconds, closing on Escape or a click outside. Two tabs. Agents: each agent's logo and name, how it's connected (MCP, hooks or detected), whether it's working, idle or braked, how Mewndo monitors it (Guarded, Watched or Data only), and a one-line live activity. Connections: each protected folder, and later each connected account, with small agent bubbles showing which agents are using it, pulsing while active. Every bubble shows its confidence: solid when exact, outlined with "likely" on hover when guessed, grey question mark when unknown. Use only local data for now. Use agent and app logos only to identify them, following each company's brand rules, with a neutral initial badge as fallback. Commit with the message "agents and connections panel" and push.

P3.5 Voice commands

Add hold-to-talk on the bar's mic. Put speech-to-text behind an interface so we can swap providers; start with a free local speech model that runs on the PC. Turn what I say into one of these intents: brief, stop an agent, freeze everything, undo what an agent did in the last N minutes, resume, and what changed today. Always show what will happen and ask me to confirm before undo or freeze. Anything unclear asks "did you mean" instead of acting. Commit with the message "voice commands" and push.

P3.6 Shortcut defaults

Wispr Flow uses a hold on Control plus Alt for dictation, which collides with our Control Alt Z and Control Alt B defaults. Change the default shortcuts to combinations that don't start with Control plus Alt, check them with the existing Test button, and on first run offer free alternatives if a default is taken. Commit with the message "shortcut defaults" and push.

Phase 4: Decision Service with Clef (spec §26, §29) and Kaggle

P4.1 Decision Service on Workers AI (free)

In the cloud folder, create a Cloudflare Worker called mewndo-decide using the Workers AI binding with the model at cf slash cloudflare slash clef-flash. It takes a state and up to 64 questions of type noul, choice or score, as in spec section 29.5, and returns the probabilities. Use one interface so we can later switch to Clef, Jev or a self-hosted Clef-flash. Add caching of identical requests for a few minutes. Count neurons used per day and stop calling the model at 9,000 neurons, returning a "use rules" answer instead, so we stay inside the free 10,000. Log latency for every call. Protect it with a secret token. Deploy it on the free plan and give me the URL. Commit with the message "decision service" and push.

You should see: a deployed Worker URL and a test call returning probabilities.

P4.2 Deadlines and fallback in the core

In mewndo-core, call the decision service only for cases the rules can't decide. Batch every question for an action into one request. Use the deadlines from spec section 29.4: 60 milliseconds for Guard hooks, 100 for sends, 200 for Heal, 150 for voice. Send a second identical request if the first hasn't answered halfway through the deadline, and take whichever answers first. If the deadline passes, the rules decide: allow harmless actions, deny destructive ones, hold sends. Record deadline met, fallback used and latency for every decision. Commit with the message "deadlines and fallback" and push.

P4.3 Measure real latency

Write a script that sends 500 realistic decision requests to the decision service from this PC and reports p50, p95 and p99 latency, the fallback rate and neurons used. Save the results to docs/latency.md. Don't change any code; just measure. Commit with the message "latency measurement" and push.

You should see: docs/latency.md . Paste the numbers to me; they decide whether you'll ever need self-hosted GPUs (spec §29.6).

P4.4 Kaggle notebook: evaluation set

In the notebooks folder, write a Kaggle notebook that builds Mewndo's evaluation and training set for drift decisions. Generate realistic examples: a brief, a planned agent action, and the correct label of allow, deny or ask, covering every rule in spec section 24.1 plus hard ambiguous cases, and email checks like wrong recipient, look-alike domains and wrong attachments. Aim for at least 5,000 examples with a held-out test split. Save it as a Kaggle dataset. Tell me exactly how to upload and run it on Kaggle. Commit the notebook with the message "eval set notebook" and push.

P4.5 Kaggle notebook: benchmark Clef-flash

Write a second Kaggle notebook that runs the evaluation set against Clef-flash through our decision service and reports accuracy per rule type, calibration, and how often a probability between 0.3 and 0.8 would ask the user. Optionally load the open Clef-flash weights in 4-bit on Kaggle's GPUs to compare accuracy, not speed. Keep within Kaggle's free GPU hours. Commit with the message "clef benchmark notebook" and push.

P4.6 Kaggle notebook: fine-tune Laya for the PC

Write a third Kaggle notebook that fine-tunes Laya, the small open decision model, on our training set, calibrates its confidence on the validation split, and exports it to ONNX. Report test accuracy next to Clef-flash's. Base Laya is near random without fine-tuning, so only ship it if its accuracy is close to Clef-flash's on our test set. Then, in mewndo-core, add an optional local judge that runs the ONNX model on the CPU, used before the cloud decision service when enabled. Measure its latency on this PC. Commit with the message "local judge" and push.

You should see: a table comparing Laya and Clef-flash accuracy, and Laya's latency on your PC.

Phase 5: MCP Connect and Send Guard (spec §22, §31.5)

P5.1 Local MCP server

Add a local MCP server inside mewndo-core, over stdio, using the official MCP SDK. Tools from spec section 22.3: mewndo_status, create_save_point, list_changes, request_delete, get_project_card and append_progress. Undo is not exposed to agents. Write tests that start the server and call every tool. Commit with the message "local mcp" and push.

P5.2 Connect page

Build the Connect page from spec section 22.2 in Settings: one card per agent showing Connected, Guarded or Not found. Buttons install the MCP server and the Guard hooks: claude mcp add for Claude Code, codex mcp add or a config.toml block for Codex, and the Cursor install deeplink for Cursor. Always show the exact change first, keep a backup, and make installing twice change nothing. Add uninstall. Check each tool's current documentation before writing config. Commit with the message "connect page" and push.

You should see: one click connects and guards each local agent, and the Agents tab shows them.

P5.3 Hosted MCP on Cloudflare

In the cloud folder, build the hosted Mewndo MCP server on Cloudflare Workers using the Agents SDK McpAgent with Streamable HTTP, OAuth 2.1 with dynamic client registration and refresh tokens, and per-tool scopes. Store each user's state in a Durable Object. Expose the same tools as the local server, plus send_email and request_delete, which go through holds. Deploy on the free plan and give me the URL. Then write the Connect cards for claude.ai custom connectors, ChatGPT developer mode, Grok custom connectors and Gemini, each

with the URL and the honest limits from spec section 22.2. Commit with the message "hosted mcp" and push.

P5.4 Link the desktop app to the cloud

Connect the desktop app to the user's Durable Object over a WebSocket, so cloud holds, approvals and cloud-agent activity appear on the bar and the up-arrow panel, and approvals from the bar reach the cloud. File contents and names never leave the PC. Reconnect automatically. Commit with the message "desktop cloud link" and push.

P5.5 Send Guard

Build Send Guard from spec section 28.4, Flow D. When an agent calls `send_email`: run the rules in under 5 milliseconds (first-time recipient, look-alike domains, too many recipients, external with attachments, secrets and personal data, leftover placeholders); check the recipient against the brief, the thread and the Continue card; then ask the decision service every question in one call. Release only if every rule passes and the model's allow probability is at least 0.97; otherwise hold with a countdown chip on the bar and the phone, and never send without a verdict. Send through Gmail with only the send scope. Log every decision. Write tests for each hold reason and for a deadline timeout. Commit with the message "send guard" and push.

You should see: a held email showing up as a chip on the bar, and an approved one sending.

Phase 6: Heal the cloud (spec §25.3, §28.4, §31.6)

P6.1 Google setup

Walk me step by step through creating a Google Cloud project for Mewndo in Testing mode: the OAuth consent screen, adding me as a test user, the Gmail and Drive APIs, and a Pub/Sub topic that Gmail can publish to, pushing to a Worker URL. Explain that tokens expire every 7 days in Testing mode and up to 100 test users can be added. Don't write code yet; give me the clicks.

P6.2 Gmail Rewind and Heal

Build the Gmail worker on Cloudflare: renew watch every day with a scheduled trigger, receive Pub/Sub pushes, and always reconcile with history list, since Gmail drops notifications above one per second. Journal each message's labels and trash state. When the Heal rules from spec section 24.4 say a change is out of scope, put messages back with one batch modify call, restoring the exact labels. Be honest in the app that permanently deleted mail can only come back from a copy Mewndo made before. Test with my test account: trash 1 message and then 100, and measure the time to restore. Commit with the message "gmail heal" and push.

You should see: measured restore times, aiming for 2 to 10 seconds for one message and under 15 seconds for 100.

P6.3 Drive Rewind and Heal

Build the Drive worker: changes watch, renewed before it expires, followed by changes list; journal trash state, parent folders and revisions; untrash and restore the original folder when Heal says so. For files in folders I mark as watched, keep a copy in Cloudflare R2 within the free tier, and warn me before going over it. If a file was permanently deleted, re-upload it

from the copy and tell me it has a new ID and its share links need redoing. Test with a trashed file and a permanently deleted one. Commit with the message "drive heal" and push.

P6.4 Notion Rewind and Heal

Build the Notion worker: webhooks plus polling on last edited time, block snapshots for pages I share with Mewndo, and restore with `in_trash` false using the current API version, then content from snapshots. Respect about three requests per second. Be honest in the app that Notion notifications can take a minute or more, so Notion undo is about one to two minutes unless the change went through Mewndo's MCP. Commit with the message "notion heal" and push.

P6.5 Real data in the Connections tab

Feed cloud accounts into the up-arrow panel's Connections tab: each account's protection state and the agent bubbles using it. A bubble is exact when the action came through Mewndo's MCP, likely when it's matched by time to a known agent session, and a grey question mark otherwise. Tapping an account shows recent changes grouped by agent, with Undo. Commit with the message "connections live" and push.

Phase 7: Flight Recorder (spec §30, §31.6)

P7.1 Local ledger

Build the Flight Recorder in `mewndo-core` from spec section 30.2: every guard decision, heal, hold, approval, save point and restore becomes an event with agent, vendor, brief hash, action, target, before and after hashes, decision, model probabilities, approver and restore receipt. Chain events by

hash and sign them with a device key stored in Windows Credential Manager. Restore receipts include the verification result. Write tests that tamper with one event and drop one event, and check both are detected. Commit with the message "flight recorder" and push.

P7.2 Anchoring, export and verifier

Every hour, compute a Merkle root of new events and send it to the cloud, and also publish the roots to a public GitHub repo as a simple transparency log. Add an export of the ledger and a small open-source command-line verifier that checks an export without trusting Mewndo. Test the verifier on a clean folder. Commit with the message "ledger anchoring" and push.

Phase 8: Release

P8.1 Test gate in CI

Set up GitHub Actions on a Windows runner that runs unit tests, the reliability suite, the guard test harness and the benchmarks on every push, and fails if any restore check fails. Commit with the message "ci" and push.

P8.2 Installer and updates

Update the installer so it installs mewndo-core and the desktop app together, starts the core at login, and supports signed auto-updates once we have a code-signing certificate; until then, leave a clear note in the README. Commit with the message "installer v1" and push.

P8.3 Honest docs

Update the README and the "What Mewndo can and can't undo" page to match spec section 28.10 exactly: what to say and what never to say, with the real numbers from docs/benchmarks.md and docs/latency.md. Commit with the message "v1 docs" and push.

Not in these prompts (needs money, approvals or a company account)

Item	Why it's left out	When
Mail Guard relay for Google Workspace or Microsoft 365 (spec §28.4 Flow E)	Needs a company tenant's admin and a server with fixed IPs	With the first company customer
Google verification and the restricted-scope security assessment	Paid and takes weeks; Testing mode covers 100 users	Before public launch
Code-signing certificate	Paid	Before public launch
Self-hosted Clef-flash for a firm ~39 ms (spec §29)	About \$1,300 per GPU per month	When P4.3 latency or usage justifies it
Kernel components (minifilter, Endpoint Security, eBPF)	Driver signing and Apple approval	Spec §28.9 phase 3

Helper prompts

When something breaks:

Something is wrong: [what you clicked and what happened].
Find the cause first and tell me before changing code. Then fix it, add a test that would have caught it, run everything, commit and push.

When tests fail:

Tests are failing. Show me which ones and why. Fix the code, not the tests, unless a test is genuinely wrong; if so, explain why before changing it.

When the agent gets lost:

Stop. Read docs/spec.md sections 21 to 31 and CLAUDE.md again. Tell me in five lines what we're building in this step and what you've done so far, then continue.

Starting a fresh session:

We're building Mewndo, M-E-W-N-D-O. Read CLAUDE.md, docs/spec.md and the git log for the last 20 commits. Tell me which prompt in Phase [N] we're on and what's left, then wait.

Sources

- [Workers AI pricing and free allocation](#)
- [Clef on Workers AI \(changelog\)](#)
- [Clef-flash measured on a 16 GB Mac](#)
- [Cloudflare: MCP, auth and Durable Objects free tier](#)
- [Kaggle free GPU quota guide \(2026\)](#)

- [Google OAuth Testing mode limits](#)